Proceedings of CONF-SPML 2026 Symposium: The 2nd Neural Computing and Applications Workshop 2025
DOI: 10.54254/2755-2721/2026.TJ29923

The Collaborative Cooperation of Multi-model Fusion and Large Language Models in Stock Prediction

Chunyi Zhang

School of Chemical Engineering, Central South University, Changsha, China
Zhangchunyi2006@163.com

Abstract. With the rapid growth and increasing complexity of financial market data, stock price prediction encounters issues such as significant noise, non-stationary data, and diverse information sources, thus rendering traditional quantitative models inadequate for capturing market trends and extreme events. This paper investigates stock market multimodel fusion approaches, emphasizing the combined use of deep learning, reinforcement learning, and large language models (LLMs) in stock price prediction. Specifically, it includes historical prices, technical indicators, financial news, and company announcements. And the methods involve hybrid deep learning modules (CNN, LSTM, Transformer), reinforcement learning agents based on proximal policy optimization, and LLMs for processing unstructured text, to integrate multimodal data and enhance the prediction interpretability. The results show that multimodal fusion can simultaneously detect short-term variations, long-term patterns, and overall trends. The introduction of LLMs greatly boosts factor attribution and strategy explainability, improving prediction accuracy, market adaptability, and decision-making, while providing insights for portfolio management, risk control, and automated trading.

Keywords: Reinforcement learning, Deep learning, Trading Agents, Stock Price Prediction, Large Language Models

1. Introduction

The stock market, as a key component of the national financial infrastructure, has fluctuations that affect both the efficiency of resource allocation and the stable functioning of the macroeconomy. In recent years, with the increasing complexity of the market and the rising frequency of transactions, stock price prediction methods based on traditional econometric models have gradually revealed some significant limitations, particularly in dealing with sudden market events. Traditional models often lack the necessary flexibility and adaptability in such scenarios. They struggle to account for dynamic shifts in market conditions, risk constraints, and transaction costs, and tend to perform poorly when predicting extreme market events, like “black swan” occurrences [1]. Besides, despite the application of deep learning in stock market prediction, its “black-box” nature still restricts its practical operability in high-risk financial decision-making. Accordingly, the HARLF framework reveals the effective integration of various technologies in financial portfolio optimization, offering valuable insights and methodological references for stock price prediction [2]. On this basis, this paper proposes a hybrid prediction and decision framework combining deep learning, reinforcement

© 2025 The Authors. This is an open access article distributed under the terms of the Creative Commons Attribution License 4.0 (https://creativecommons.org/licenses/by/4.0/).

DOI: 10.54254/2755-2721/2026.TJ29923

learning, and large language models (LLMs), aimed at improving the accuracy and adaptability of stock market prediction. By integrating deep learning models for feature extraction from structured data, utilizing reinforcement learning to optimize trading strategies, and employing LLMs for the analysis and interpretation of unstructured text, it addresses the limitations of traditional models in complex financial environments, while improving model clarity and risk management.

2. Stock price prediction based on traditional deep learning models

2.1. Basic principles and feature extraction

In stock price prediction, deep learning extracts time series features at various levels, thus capturing micro-fluctuations, long-term trends, and global dependencies. Among these models, convolutional neural networks (CNNs), long short-term memory networks (LSTMs), and Transformers are the most commonly utilized, each excelling in handling specific time scales and types of information.

By using convolution kernels, CNN identifies short-term patterns via the weighted summation of local windows. However, some studies have shown that although traditional pooling operations can reduce dimension and enhance translation invariance, they may miss crucial local information. In contrast, non-pooling CNN (NPCNN) can retain more local structures, thus improving prediction accuracy [3]. Meanwhile, LSTM controls information memory and forgetting through input gates, forgetting gates, and output gates, capable of capturing long-term dependencies and filtering noise, making it suitable for handling cyclical fluctuations and trend changes in stock prices. In addition, the Transformer captures global dependencies via self-attention, and multi-head attention helps the model focus on dependencies across time spans, boosting its capture of complex trends. In practice, CNN identifies short-term patterns, LSTM models long-term relationships, and Transformer merges global information for multi-scale features. Existing studies reveal that multi-scale feature splitting can improve prediction accuracy [4]. This strategy enhances input, improving feature capture across time scales and boosting accuracy and robustness.

2.2. Parameter optimization and particle swarm optimization algorithm

The hyperparameters of the model markedly affect prediction performance; thus, their optimization is essential for constructing a high-precision stock price prediction model. Hyperparameters include network structure parameters as well as training control parameters. These configurations directly influence the convergence and generalization capabilities of the model. To improve the efficiency of hyperparameter search and minimize the costs and subjectivity associated with manual trial and error, several optimization strategies are commonly employed, including grid search, random search, Bayesian optimization, and particle swarm optimization (PSO). Among these, PSO mimics group collaboration to identify the global optimal solution, with each set of hyperparameters represented as particles, the fundamental entities of the algorithm. Each particle represents a potential solution to the optimization problem and iteratively updates its position and velocity within the search space [5]. Its fitness function is usually evaluated based on the root mean square error (RMSE) of the validation set to evaluate the effectiveness of the particle's current position [6].

$$\mathrm{RMSE} = \sqrt{\frac{1}{N_{\mathrm{val}}} \sum_{i=1}^{N_{\mathrm{val}}} \left(y_i - \hat{y}_i\right)^2} \tag{1}$$

In each iteration, the particles' position and velocity are updated based on the following rules.

DOI: 10.54254/2755-2721/2026.TJ29923

$$v_i^{t+1} = \omega v_i^t + c_1 r_1 \left(p_i - x_i^t\right) + c_2 r_2 \left(g - x_i^t\right) \tag{2}$$

$$x_i^{t+1} = x_i^t + v_i^{t+1} \tag{3}$$

where the particle's position x_i and velocity v_i are updated based on the individual historical optimal position p_i and the global best position g, while also introducing random perturbations to increase search diversity. Through multiple iterations, the particles gradually converge to the global optimum, yielding the optimal hyperparameters for the model. Compared to traditional grid search, PSO converges more quickly in high-dimensional spaces, balancing global exploration and local exploitation, which boosts efficiency and lowers the risk of getting stuck in local optima. Therefore, in stock price prediction tasks, hyperparameter configurations optimized by PSO can significantly improve model training stability and prediction accuracy.

2.3. Model architecture and training methods

In stock price prediction, key aspects of model training include sample construction, loss function selection, training control, and evaluation. Among these, the sliding window method is a commonly used sample construction technique, generating input samples by extracting feature sequences from consecutive trading days. The selection of window length typically balances historical data volume and computational efficiency. The p-Huber loss function is dynamically adjusted according to the magnitude of the error and has certain robustness against outliers. Its definition is as follows.

$$\phi^{p-\text{Huber}}\left(y, f\left(x\right)\right) = \begin{cases} \left(y - f\left(x\right)\right)^2, & \left|y - f\left(x\right)\right| < \delta, \\ \frac{2\delta^{2-p}}{p}\left|y - f\left(x\right)\right|^p - \frac{2-p}{p}\delta^2, & \left|y - f\left(x\right)\right| \geq \delta, \end{cases} \tag{4}$$

where f denotes the predicted value of the model, y refers to the true value, and δ represents the threshold parameter that primarily controls the inflection point of the p-Huber loss function. The parameter p mainly controls the concavity/convexity of the Huber loss function, governing the switch between the Huber loss function and the L2 loss function [7].

The training process requires control and optimization, including optimizer choice, learning rate adjustment, and overfitting prevention. Adam or its variants are commonly used to maintain stable gradient updates, while also combining learning rate scheduling and early stopping mechanisms can improve training efficiency and reduce the overfitting risk. Besides, model evaluation is conducted with training, validation, and test sets to fairly reflect its performance in stock price prediction.

3. Trading decision model based on proximal strategy optimization

3.1. Algorithm principles and method motivations

Proximal Policy Optimization (PPO) is a policy gradient method that stabilizes training by limiting policy changes. In high-dimensional and noisy financial environments, this mechanism helps reduce the risk of individual trading decisions, thus making it commonly used in stock price prediction and trading strategy learning. When modeling stock price prediction as a sequential decision-making problem, PPO combines market states, trading actions, state transitions, and reward functions. The state $\mathrm{s_t} \in \mathrm{S_B}$ represents the observed information at time t, including historical price sequences, technical indicators, and pre-processed macro event embedding vectors. The action a_t of the agent

DOI: 10.54254/2755-2721/2026.TJ29923

refers to executable trading operations, like buying, selling, or holding, while the state transitions in the environment are driven by historical data, with the state transition probabilities $P(s_{t+1}|s_t, a_t)$ reflecting the real market shifts. The reward $r(s_t, a_t)$ is based on risk-adjusted returns, guiding the agent to optimize its trading strategy. To improve training stability, PPO constrains the magnitude of policy updates, and its objective function is defined as follows.

$$J(\theta)=E_t\left[\min\left(r_t(\theta)A_t,\ clip(r_t(\theta),1-\epsilon,1+\epsilon)A_t\right)\right] \tag{5}$$

where $r_t\left(\theta\right)=\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ refers to the strategy proportion, and A_t means the advantage function, typically computed using Generalized Advantage Estimation (GAE) [8]. This mechanism allows PPO to stably learn trading strategies in high-dimensional, non-stationary market environments, improving the robustness of the strategy and long-term performance. Thus, PPO has become the preferred method for tackling complex sequential decision-making challenges in financial trading.

3.2. State representation and information fusion

The construction of the state space directly affects the reinforcement learning agent's ability to perceive market information and make decisions. A typical multi-source state representation usually combines numerical features with textual events, creating a fuller market representation. Textual events provide important supplementary signals beyond numerical features, including company announcements, news reports, and macroeconomic policy information. These signals can reflect changes in investor sentiment and market expectations. In research, pre-trained language models are commonly used to encode text data into high-dimensional semantic vectors, which are then aligned and fused with numerical features, enabling cross-modal information embedding. This approach enhances the model's responsiveness to sudden events or market shifts, improving the adaptability and robustness of the strategy. Besides, previous studies have also explored graph neural networks or multi-channel encoding frameworks to strengthen the interaction modeling capabilities between different modalities. Therefore, multi-modal fusion improves state representation, providing a more reliable foundation for reinforcement learning-based trading decisions.

3.3. Reward design and risk control

The reward function influences the agent's learning and optimization focus in trading strategies. In practice, it considers three factors: returns, transaction costs, and risk. First, the return component reflects the profitability of the portfolio over a given period, incentivizing the strategy to maximize returns. The transaction cost component penalizes buying and selling actions, ensuring the strategy controls excessive trading costs while optimizing returns. Beisdes, the risk component limits return volatility and downside risks, boosting the strategy's robustness in uncertain market conditions. The risk component constrains the volatility of returns and downside risks, thus enhancing the strategy's robustness in uncertain market environments. Based on these principles, a dynamic, multi-objective reward can be used to balance returns and risk control.

$$R_t=R_{profit}-\lambda_c\cdot C_{trade}-\lambda_v\cdot\sigma_t \tag{6}$$

DOI: 10.54254/2755-2721/2026.TJ29923

where R_{profit} means the portfolio's return over the period, C_{trade} refers to the transaction cost penalty term, and σ_t is the volatility risk penalty based on the period's return (volatility, downside risk), with the hyperparameter λ_c and λ_v employed to balance the different preferences between returns and risk [9]. By considering returns, transaction costs, and risk, the multi-objective reward not only seeks profit but curbs excessive trading and limits volatility, thus enhancing the model's decision-making robustness in complex market environments. This mechanism enables the agent to optimize trading behavior in high-dimensional and volatile financial environments, improving the strategy's long-term returns and risk control capabilities.

3.4. Overall architecture and strategy optimization

The PPO agent, based on the Actor-Critic architecture, includes policy generation (Actor) and value evaluation (Critic), forming the core of trading decisions in complex markets. The system consists of an environment simulator, an agent, and an experience replay buffer. The environment simulator provides state transitions and reward signals, while the agent generates trading actions based on the current state. Besides, the experience replay buffer stores interaction trajectories, supporting offline learning and enhancing sample efficiency and training stability. The training process is divided into two stages: supervised pre-training and reinforcement learning fine-tuning. During the supervised pre-training stage, historical data and return labels train the Actor network, providing knowledge for strategy initialization. In the reinforcement learning fine-tuning phase, the agent optimizes the strategy through a "trial-and-error-feedback" loop, while the Critic network minimizes temporal difference errors to learn the value function. To optimize the PPO objective, stochastic gradient ascent is used with gradient clipping and early stopping on the validation set to ensure stability and prevent overfitting. Moreover, by analyzing state variables with SHAP, feature attribution enhances strategy clarity and performance, enabling robust decisions in volatile markets.

4. Framework for enhancing prediction interpretability based on large language models

4.1. Interpretability bottlenecks and issue assessment

Given their use of deep learning and reinforcement learning, stock price prediction models are often black-boxes, lacking feature interpretation, untraceable policy iteration, and making it difficult to identify error sources. These issues not only limit the model's interpretability and reliability but also increase the difficulty of strategy optimization and risk control. Specifically, at the feature level, the model struggles to clarify the contribution of each input feature to the prediction result, making it difficult for investors to understand the basis of decisions; during the policy iteration process, the agent's decision updates are untraceable, making the strategy optimization logic difficult to verify; regarding error sources, the model's prediction biases in different market environments are difficult to explain, posing challenges to risk management. To address these issues, LLMs can be introduced to enhance interpretability. With their strong semantic understanding and generation capabilities, LLMs can convert the model's numerical decision-making basis like feature importance, attention weights, and policy values into structured natural language descriptions. Besides, they use past case matching and counterfactual reasoning to provide intuitive, reliable decision explanations [10].

DOI: 10.54254/2755-2721/2026.TJ29923

4.2. Multimodal input and feature fusion

In stock price prediction tasks, data scarcity and insufficient cross-modal information fusion often limit model performance. Financial data modalities, including price sequences, technical indicators, macroeconomic texts, and news sentiment, differ in dimensions and features, limiting the model's ability to integrate information. Thus, a multimodal fusion mechanism can integrate information from different sources to enhance representation capability. Specifically, SH-Mix (Saliency-Aware Hierarchical Multimodal Mixup) adopts a saliency-aware hierarchical multimodal Mixup strategy. This method first weights the features of each modality based on saliency scores to identify key predictive signals, then performs modality Mixup within a hierarchical structure to enhance features and expand the data. By mixing input and feature spaces, it captures cross-modal dependencies and mitigates training bias from data scarcity in a single modality. Therefore, the enhanced multimodal representations are then fed into a LLM, encoding latent relationships between numerical sequences, event texts, and technical indicators into interpretable semantic vectors, boosting the accuracy and robustness of stock price predictions [11]. Moreover, the saliency-aware mechanism in hierarchical Mixup improves noise suppression, ensuring stable predictions in volatile markets/

4.3. Decision logic and interpretable generation

In stock price prediction, LLMs convert multimodal features and predictions into clear outputs, improving model transparency and reliability. Studies indicate that by analyzing key intermediate variables, LLMs offer structured explanations for trading strategies, boosting decision verifiability. The interpretability of decision logic usually depends on multimodal features, commonly using a three-tier framework: “rule-based identification, scenario matching, and error calibration.” At the base level, important temporal signals are identified via rules or thresholds; the middle layer adjusts strategy weights based on historical scenarios; and the top layer optimizes feature combinations using error feedback for dynamic strategy calibration [12]. This process combines multi-source data and adapts prediction strategies. In terms of feature contribution and historical case comparison, the impact of input features on prediction results can be quantified using methods like control variables or gradient attribution (e.g., SHAP, Integrated Gradients). These methods, combined with historical similar cases, produce structured reports that outline feature roles and related cases, thus enhancing explanation verifiability. In counterfactual analysis, LLMs can create hypothetical scenarios, such as predicting outcome changes without a specific policy, and validate results by comparing with historical cases. This multimodal fusion mechanism offers verifiable, intuitive strategy explanations, aiding trading decision analysis.

5. Conclusion

This paper presents a fusion framework combining deep learning, reinforcement learning, and large language models for stock price prediction. The results indicate that CNN, LSTM, and Transformer models excel in temporal feature extraction, while reinforcement learning algorithms such as PPO enhance strategy adaptation and risk-awareness. LLMs boost interpretability via feature attribution, case comparison, and counterfactual analysis, increasing prediction transparency. By combining perception, decision-making, and explanation, the “deep feature extraction - reinforcement decision optimization - semantic explanation generation” framework leads financial prediction. However, challenges remain, mainly the limited stability and generalization of reinforcement learning. Future

DOI: 10.54254/2755-2721/2026.TJ29923

work could improve financial decision systems' efficiency and transparency through cross-modal alignment and counterfactual reasoning.

References

[1] Wang, L., Deng, X., Ge, P., Dong, C., Bethel, B.J., et al. (2022). CNN-BiLSTM-Attention model in forecasting wave height over South-East China Seas. Computers, Materials & Continua, 73(1), 2151-2168.
[2] Coriat, B., & Benhamou, E. (2025). HARLF: Hierarchical reinforcement learning and lightweight LLM-driven sentiment integration for financial portfolio optimization. arXiv. https: //arxiv.org/abs/2507.18560
[3] Liu, S., Ji, H., & Wang, M.C. (2020). Nonpooling convolutional neural network forecasting for seasonal time series with trends. IEEE Transactions on Neural Networks and Learning Systems, 31(8), 2879-2888.
[4] Yang, C., et al. (2023). Human action recognition using key-frame attention-based LSTM networks. Electronics, 12(12), 2622.
[5] Zhang, Z.Z., Zhang, Y.Y., & Zhu, Y.Q. (2025). A method forsearching ESN network architecture based on multi - objective optimization. Control Engineering of China, 1-12.
[6] Chai, T., & Draxler, R.R. (2014). Root mean square error (RMSE) or mean absolute error (MAE)? - Arguments against avoiding RMSE in the literature. Geoscientific Model Development, 7(3), 1247-1250.
[7] Yu, B.T. (2020). Research on p-Huber loss function and its robustness. Advances in Applied Mathematics, 9, 2283.
[8] Zheng, H., Gao, N., Cai, D., Jin, S., & Matthaiou, M. (2025). UAV individual identification via distilled RF fingerprints-based LLM in ISAC networks. arXiv. https: //arxiv.org/abs/2508.12597
[9] Yang, F.J. (2025). Research on stock price prediction method based on deep learning. Dalian Jiaotong University.
[10] Brown, T.B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. Advances in neural information processing systems, 33, 1877-1901.
[11] Jain, S., Chhabra, P., Neerkaje, A.T., et al. (2024). Saliency-aware interpolative augmentation for multimodal financial prediction. The 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024), 14285-14297.
[12] Kang, M.J.Y., Yang, W., Roberts, M.R., Kang, B.H., & Malpas, C. B. (2025). Beyond black-box AI: Interpretable hybrid systems for dementia care. arXiv. https: //arxiv.org/abs/2507.01282